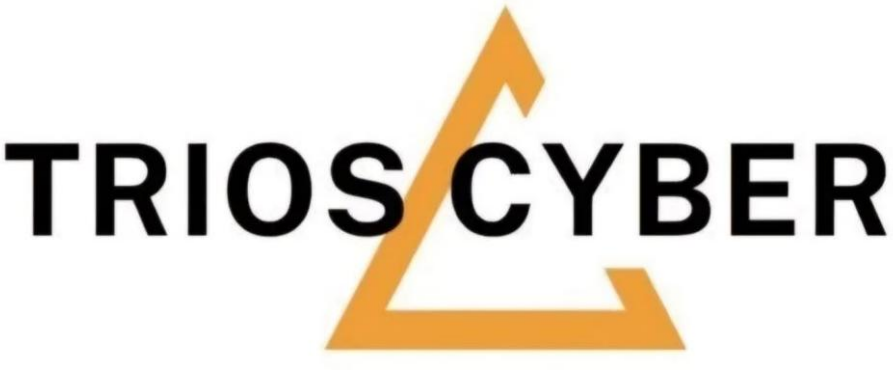


TRIOS CYBER

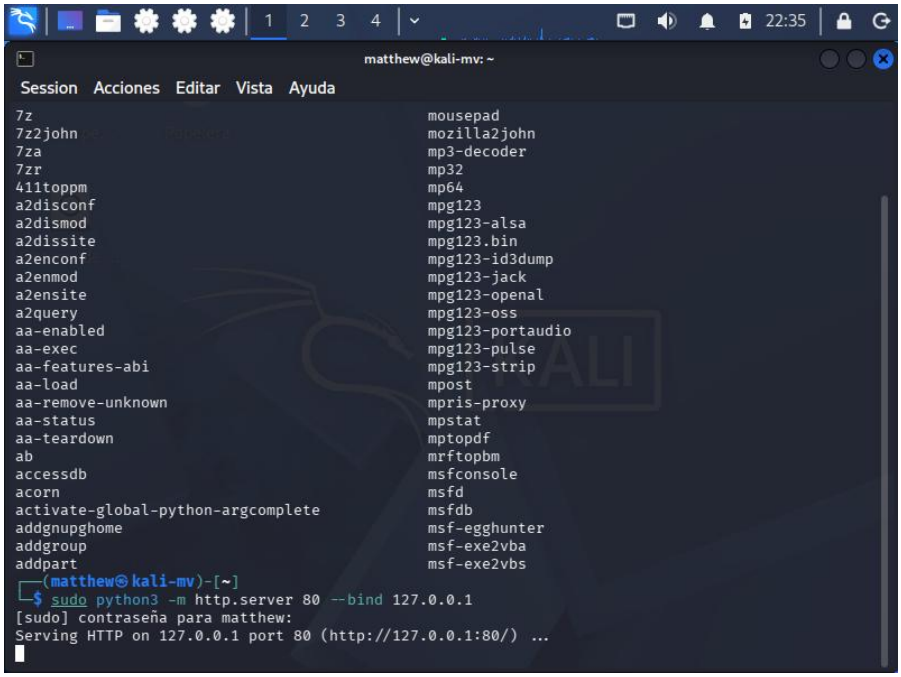
LABORATORY PRACTICE 4

TCP Connection Establishment and Teardown Analysis

Name: Matthew Josue Portilla Morejon

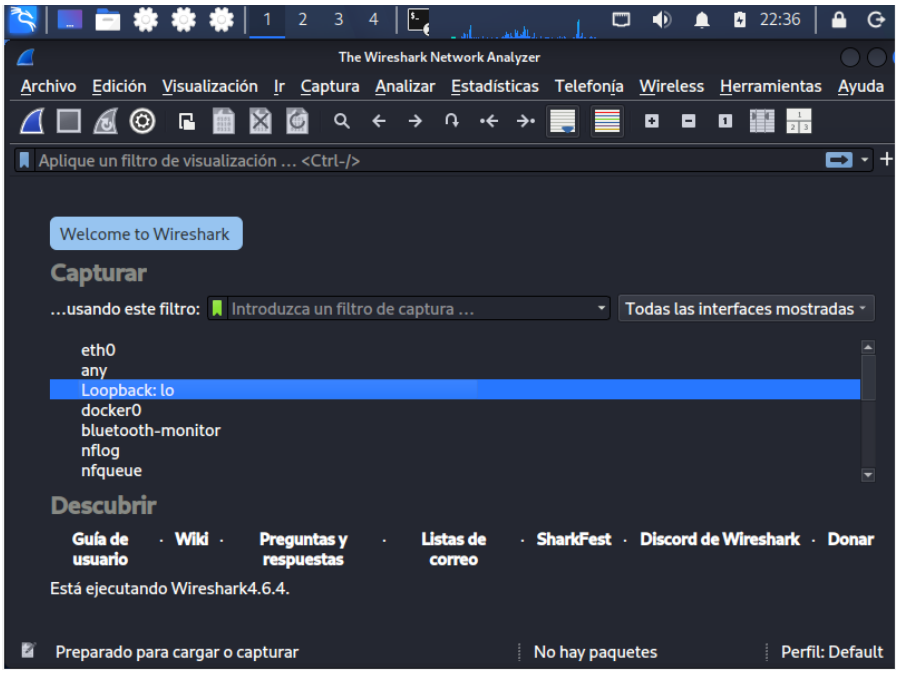
1. Local TCP Service Setup

An authorized local HTTP service was started on Kali Linux using Python and bound to the loopback address 127.0.0.1 on TCP port 80. This provided a controlled local service for generating the TCP connection required by the laboratory.



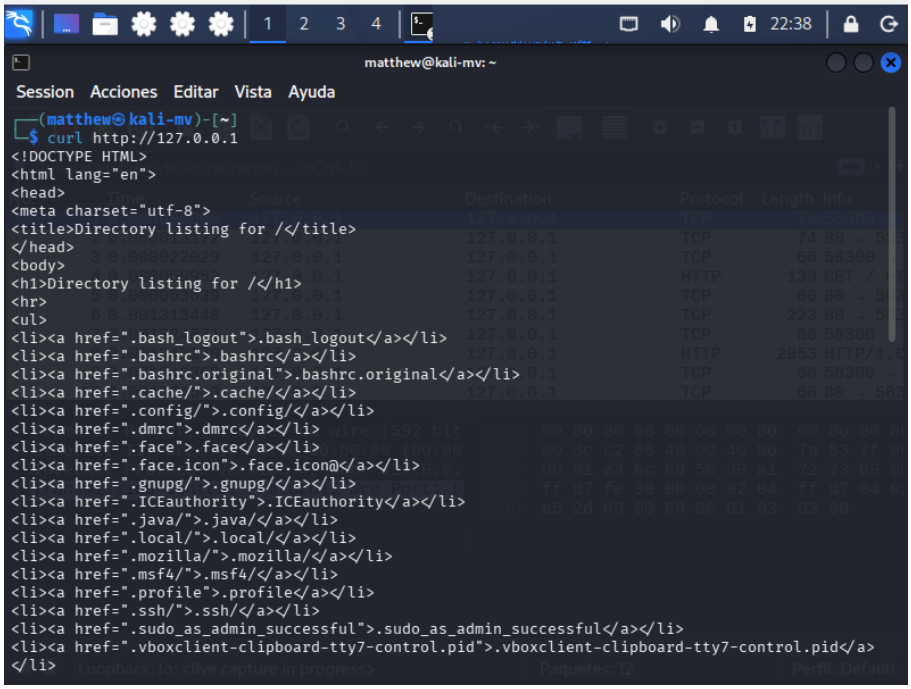
2. Wireshark Capture Preparation

Wireshark was opened on the Kali Linux virtual machine and the loopback interface (lo) was selected. This interface was used because both the client and the authorized HTTP service communicated locally through 127.0.0.1.



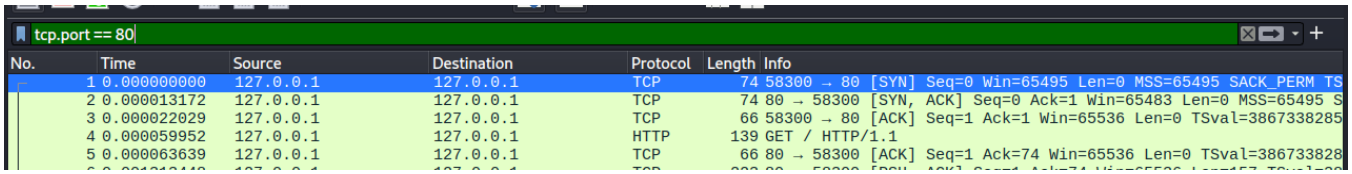
3. TCP Connection Generation

A TCP connection to the local HTTP service was generated with curl using http://127.0.0.1. The request successfully reached the local service and returned the HTTP directory response, confirming that a complete TCP session had been created for analysis.



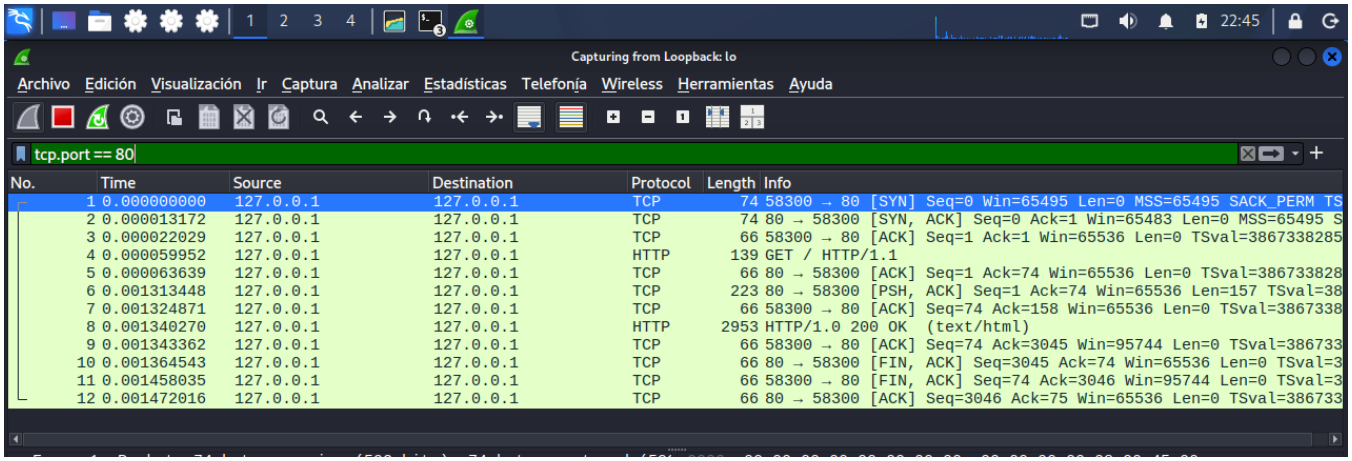
4. Three-Way Handshake Identification

The captured traffic was filtered with tcp.port == 80. The beginning of the session showed the expected TCP three-way handshake: a SYN from client port 58300 to server port 80, a SYN-ACK from port 80 back to 58300, and a final ACK from 58300 to port 80 before the HTTP request was transmitted.



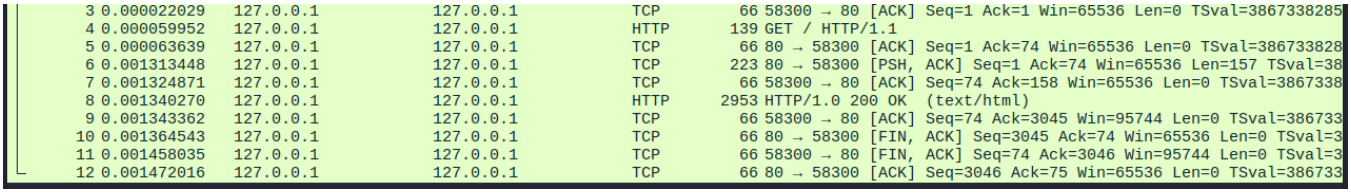
5. Source and Destination Port Analysis

The connection used 127.0.0.1 as both the source and destination IP address because the communication remained on the loopback interface. The client selected the temporary source port 58300, while the local HTTP server listened on destination port 80. The response traffic reversed these port roles from 80 to 58300.



6. TCP Connection Teardown

Connection termination packets were visible in the capture. The server sent a FIN-ACK from port 80 to client port 58300, the client replied with a FIN-ACK from 58300 to port 80, and the server completed the teardown with a final ACK. This showed the orderly termination of the local TCP session.



7. Sequence of Events Review

The captured session followed the expected TCP lifecycle. The client first initiated the connection, the server acknowledged and accepted it, and the client completed the handshake. HTTP application data was then exchanged before both endpoints closed the connection using FIN and ACK flags.

Step	Packet	Source	Destination	Behavior
1	SYN	127.0.0.1:58300	127.0.0.1:80	Connection request
2	SYN, ACK	127.0.0.1:80	127.0.0.1:58300	Server acknowledgment
3	ACK	127.0.0.1:58300	127.0.0.1:80	Handshake completed
4	HTTP GET	127.0.0.1:58300	127.0.0.1:80	Application request
5	HTTP 200 OK	127.0.0.1:80	127.0.0.1:58300	Application response
6	FIN, ACK	127.0.0.1:80	127.0.0.1:58300	Server begins teardown
7	FIN, ACK	127.0.0.1:58300	127.0.0.1:80	Client closes its side
8	ACK	127.0.0.1:80	127.0.0.1:58300	Connection closed

8. Conclusions

The TCP analysis was successfully completed using an authorized local HTTP service on Kali Linux. Wireshark captured the complete connection lifecycle, including the SYN, SYN-ACK, and ACK packets used to establish the session, the client and server port roles, and the FIN-ACK and ACK packets used during connection teardown. The results confirmed how TCP establishes, maintains, and orderly terminates a connection within a controlled local laboratory environment.